SOFTWARE CONSTRUCTION AND METHODS

STUDENT LEARNING MANAGEMENT SYSTEM

GROUP 10

Group Members:

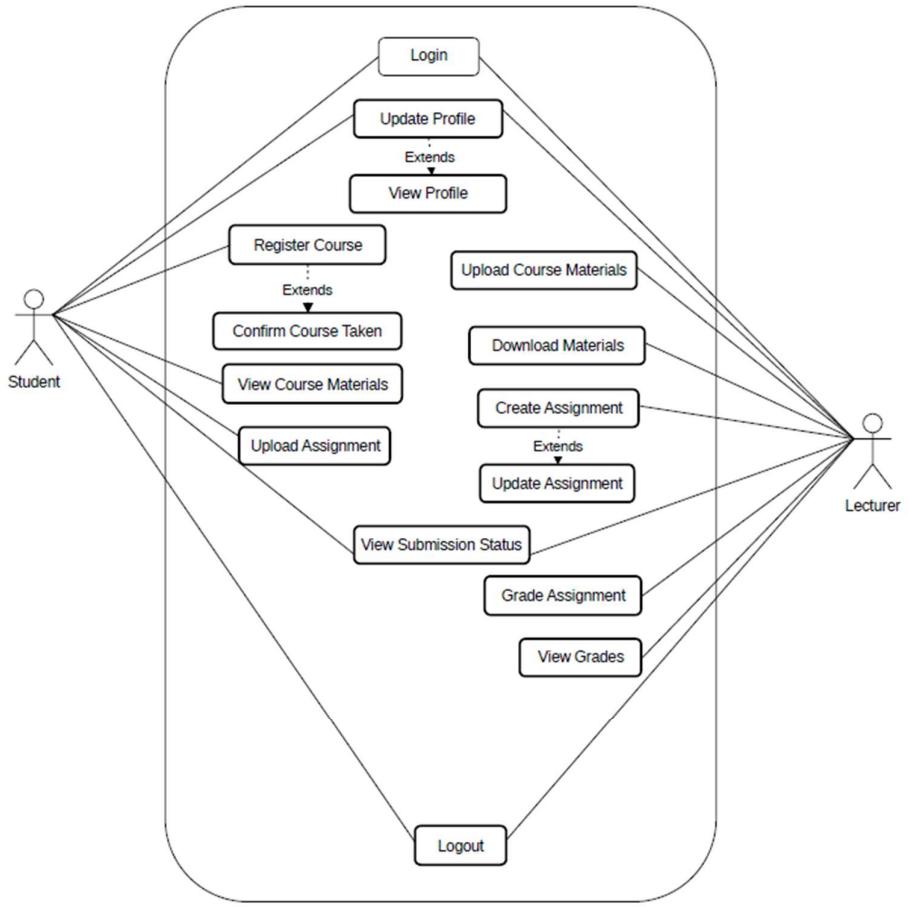
Name	Student ID
HARIZ IRFAN BIN MOHD SHAMIR	BSW01085154
MOHAMMAD DANISH DANIAL BIN MOHD ROSLI	BSW01084690
MUHAMAD IRFAN BIN ZUHAIMIN	BSW01082139

Table of contents:

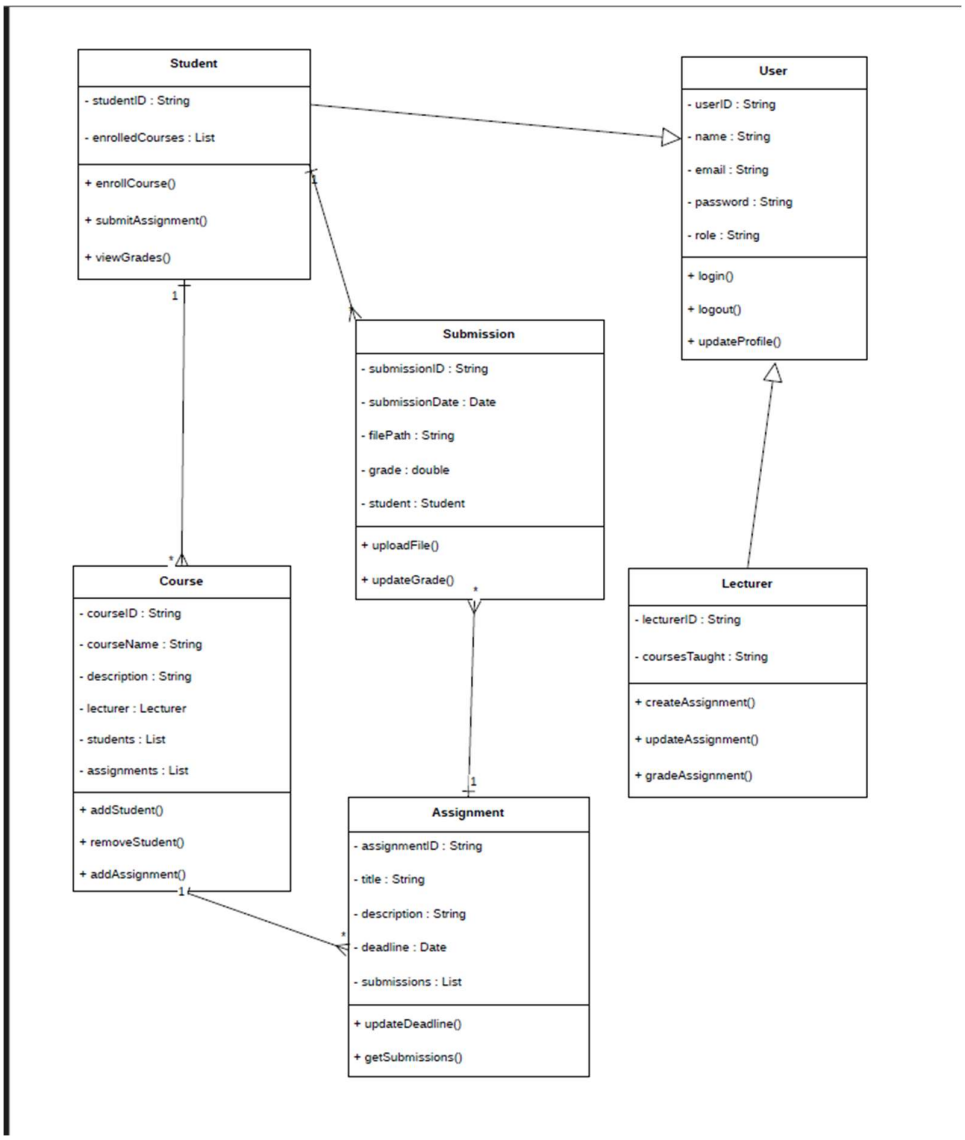
no	Section	Pages
1	Design 1.1Use Case Diagram 1.2Class Diagram	3
2	documentation formatting	5
3	coding file formatting	6
4	Define naming conventions – dictionary	7
5	Define unction and methods modularity	8
6	continuous integration rules	10

1.0 Design

1.1 Use Case Design



1.2 Class Design



2.0 Documentation Formatting

Font Style: Times New Roman.

Font Size:

-Size 12 for body text, headers, and footers.

-Size 15 for Subheadings

-Size 20 with a red color,bold and italic for section headings.

Allignment:

-Left side of the page

Page Numbering :

-the bottom right of the page (not include the cover page and table of contents)

Line spacing: 1.15

Heading format:

1.0 Main Heading

1.1 Subheading

Table format :

Cover page
Table of contents
Introduction
Conclusion

3.0 coding file formatting

3.1 Code Style Rule:

Language: Java

Indentation: 4 spaces

No tab mixing

Naming convention: Ensure any files or variables are agreed-upon to improve readability.

Max line length: 100 characters

Curly brackets on same line:

```
public void login() {
```

Comment:Added as a reminder,note and explaination for a line or the class of the code

comment format:

```
/* This method validates login*/
```

Each source files must follow;

Package Declaration

Import Statements

Class Declaration

Attributes

Constructors

Methods

Code Organization Rules.

-One public class per file.

-File name must match class name.

-Avoid unused variables/imports.

4.0 Define Naming Conventions

1.0 Purpose

Naming conventions provide standardized rules for naming system components in the Student Learning Management System (SLMS).
These standards improve:

1. **Code readability**
2. **Maintainability**
3. **Collaboration among developers**
4. **Error reduction**
5. **System consistency**

2.0 General Naming Principles

The following principles apply to all SLMS components:

1. **Names must clearly describe purpose or functionality.**
2. **Abbreviations should be avoided unless commonly understood.**
3. **Names must be meaningful and self-explanatory.**
4. **Consistent capitalization rules must be followed.**
5. **Special characters and spaces are not allowed.**
6. **English language must be used for all identifiers.**

Item	Convention	Example
Class	PascalCase	StudentAccount
Method	camelCase	submitAssignment
Variable	camelCase	studentName
Constant	UPPER_CASE	MAX_LOGIN_ATTEMPT
Database table	lowercase_with_underscore	student_record

5.0 Define Function and Methods Modularity

SLMS will be divided into independent modules where each function has a single responsibility. This approach makes the system easier to manage and ensures maintainability, scalability, and easier debugging of the SLMS system.

Module	Responsibility
Authentication Module	Login and logout
Course Module	Course creation & enrollment
Assignment Module	Assignment handling
Grading Module	Marks and feedback

1. Single Responsibility Principle

Each method will perform one specific task

• Input Methods: inputStudentData() will handle user entries only.

• Logic Methods: CalculateGrade() will focus purely on calculating the grade of the students2.High Cohesion

2.High Cohesion

Function inside the module must be related .

Module	Function
Authentication Module	login() logout() updateProfile()
Course Module	enrollCourse() addStudent() removeStudent()
Assignment Module	createAssignment() updateassignment() updateDeadline() getSubmissions()
Grading Module	gradeAssignment() viewGrades() updateGrade()

3. Method design rules

Rule	Description
Method Length	Less or equal 40 lines
Parameters	5 at most
Return Value	Value needed
Reusability	Avoid logic duplicate

6.0 Continuous Integration Rules

1.0 Purpose

Continuous Integration ensures collaborative development and prevents integration conflicts

2.0 Version Control Platform

SLMS development uses GitHub repository.

3.0 Development Workflow

1. **Pull latest develop branch.**
2. **Create feature branch.**
3. **Implement feature.**
4. **Commit changes.**
5. **Push to GitHub.**
6. **Create Pull Request.**
7. **Review before merge.**

4.0 Code must pass:

1) **No compilation errors**
2) **No warnings**

5.0 Commit message format:

[Module] Short description
Example: [Login] Added password validation